

Graduate School of Engineering and Basic Sciences

Master's Thesis

Dynamic Pricing
Neural network-based distributional constraint learning methodology for mixed-integer stochastic optimization

Master in Statistics for Data Science

Student: Daniel Losada Molina

Supervisors: Prof. Carlos Ruiz Mora

Dr. Carlos Heredia Pimienta

uc3m | Universidad Carlos III de Madrid

Abstract

This thesis applies the Distributional Constraint Learning (DCL) methodology, which has already proven effective in other scenarios, such as energy pricing. In this work, our aim is to solve a dynamic pricing problem where the demand for a product depends on its price and is uncertain. The goal is, rather than performing a point estimate, to determine optimal prices for an individual or a group of clients by modeling the demand as a conditional probability distribution. To do so, a distributional neural network is trained to predict the mean and standard deviation of demand as functions of price and contextual features. This enables the generation of demand scenarios that reflect uncertainty. These predictions are embedded into a mixed-integer stochastic optimization model using piecewise-linear constraints, allowing the incorporation of uncertainty into the pricing decision process. This is performed and evaluated on synthetic data that emulates realistic consumer behavior. The experimentation demonstrates that this methodology can be successfully transferred to the context proposed in this case study, as it yields interpretable and robust pricing. However, the main limitation that emerged is scalability with the number of clients, which motivates future work in decomposition techniques.

Contents

1 Introduction **4**

2 Demand prediction model **5**
2.1 Neural Networks 5
2.1.1 Introduction 5
2.1.2 Predicting a Distribution Instead of a Point 6
2.2 Data generation 7
2.3 Model 8

3 Optimization **11**
3.1 Optimization under Uncertainty 11
3.2 From Constraint Learning to Distributional CL 11
3.3 Dynamic pricing problem 12
3.3.1 Notation 13
3.3.2 Conceptual Model 14
3.3.3 Neural Network Embedding via Piecewise-Linear Constraints 15
3.4 Selection of number of scenarios 16
3.5 One client vs N clients 19
3.6 Solution for 5 clients 20

4 Conclusion **23**

A Appendix: Synthetic Data Generation Code **24**

B Appendix: Conceptual model definition code **25**

C Appendix: Distributional Neural Network Embedding code **26**

D Appendix: Time Distribution and Development Timeline **29**
D.1 Development Timeline 29
D.2 Estimated Time Distribution 29

E Appendix: Declaration of use of Generative AI in the Master's Thesis **30**

1 Introduction

In the industry, in many situations, the price of a product is defined or negotiated with each client. That is, the price at which client A buys a product might differ from the price at which client B buys the same product. The selection of that price by client becomes challenging as the price offered will affect the demand and even can affect how other clients behave.

The objective is always the same. Maximizing profit. But what price will do that is not clear. And is harder when it is individualized. Usually, the objective is to find the highest price that the client would pay. Dynamic pricing strategies have some advantages:

- **Increased revenue:** By adapting the prices, we can set the optimal price that will likely maximize the revenue. This can happen for example by increasing the demand, or increasing the conversion rate.
- **Competitive advantage:** Price can be reactive to the market, including the pricing of competence.
- **Improve client satisfaction:** Dynamic pricing can lead to better deals for the client, increasing their satisfaction and loyalty.
- **Improved inventory management:** Price could react to the inventory state, for example, by decreasing the price of overstocked products to sell them faster, or the other way around, increasing the price of high-demand products to slow sales and prevent stock shortages.

These strategies might be harmful. The client may not be satisfied with a policy that changes the price they receive too frequently/drastically or in a non-transparent way.

In this project, we apply the Distributional Constraint Learning (DCL) methodology to an optimization problem of decision-making under uncertainty. The objective is to determine the optimal price of a product for one or more customers under uncertainty of demand due to price elasticity. The relationship between price and demand is learned through a neural network with the capability of representing non-linear relationships that are typical in real-world environments. This model is then embedded within a stochastic optimization framework to enable robust pricing decisions. In particular, a Distributional Neural Network is trained to estimate the conditional mean and variance of the demand as a function of price and contextual information. These estimated parameters define a conditional probability distribution from which demand scenarios are drawn, leading to a more reliable decisions that take into account the uncertainty of the demand. To test the methodology, we use synthetic data that simulates real-world market environments.

2 Demand prediction model

2.1 Neural Networks

2.1.1 Introduction

Neural networks are machine learning models that are adaptive and utilized to make predictions about complicated, usually non-linear relationships between input parameters and an output. They are composed of connected layers composed by *neurons*, and their standard structure is, an input layer, one or more *hidden layers*, and an output layer.

Each neuron performs a simple computation: taking a weighted sum of the inputs it has, and a bias term, and applying this through a non-linear activation function.

The activation function is required in order to capture the non-linear relationship in the data. Without it, the network would be performing nothing more than linear transformations on the data. Activation functions allows the model to capture complex structures.

In this work, we employ the Rectified Linear Unit (ReLU) as the activation function, defined as

$$\mathrm{ReLU}(x) = (x)^+ = \max\{0, x\}$$

and implemented in PyTorch [1].

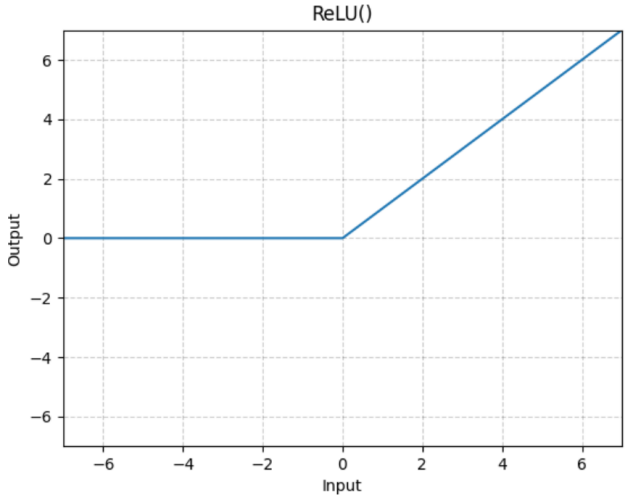


Figure 1: ReLU function.

ReLU is widely used in practice due to its simplicity and effectiveness in training neural networks. It is also a good fit for our application since its piecewise-linear nature allows for an exact representation through mixed-integer linear constraints, which makes it compatible with embedding the trained neural network directly into a mixed-integer optimization model [2].

When we say that ReLU is piecewise-linear, we mean that, even if it is not linear, it is formed by linear parts. Particularly, has two linear segments:

$$\begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$$

Training a neural network involves adjusting the weights and biases across all layers so that the network's predictions match observed data. This is typically done by minimizing a loss function using gradient-based optimization methods, such as stochastic gradient descent. In our case, Adam optimizer [3] is used. Once trained, the network can be used to make predictions for new inputs by propagating them forward through its layers.

2.1.2 Predicting a Distribution Instead of a Point

In many traditional regression tasks, neural networks are trained to predict a single value for the target variable, often corresponding to the conditional mean. However, in our setting, the target variable (demand) is uncertain and influenced by decision variables such as the price. As a result, point predictions are not enough to capture the variability in the response.

That's why we adopt a *Distributional Neural Network* (DNN) [4], a variant of the standard feedforward architecture that estimates not only the conditional mean $\hat{\mu}_y$ but also the conditional standard deviation $\hat{\sigma}_y$ of the response variable. This is based on the assumption that the conditional distribution of the target variable y given inputs x follows a normal distribution:

$$y \mid x \sim \mathcal{N}(\hat{\mu}_y(x), \hat{\sigma}_y(x)).$$

The network is a standard feed-forward network but with two outputs instead of one. Those outputs are, the predicted conditional mean $\hat{\mu}_y$ and the conditional standard deviation $\hat{\sigma}_y$. To ensure non-negativity of the standard deviation, the second output passes through a ReLU activation:

$$\hat{\mu}_y = f_{\text{mean}}(x), \quad \hat{\sigma}_y = \text{ReLU}(f_{\text{sd}}(x)).$$

At training time, we compute the mean $\hat{\mu}_y$ and the variance $\hat{\sigma}_y^2$ and apply the Gaussian negative log-likelihood (NLL) loss to fit the distribution. We use PyTorch's `GaussianNLLLoss` [5] with $\texttt{eps} = 10^{-6}$ to prevent numerical instability when the predicted variance is close to zero.

$$\mathcal{L} = \frac{1}{2}\left(\log\left(\max(\hat{\sigma}^2, \varepsilon)\right) + \frac{(y - \hat{\mu})^2}{\max(\hat{\sigma}^2, \varepsilon)}\right) + \text{const.} \tag{1}$$

This loss penalizes both inaccurate predictions and overconfident estimates with low variance.

This modeling approach enables the neural network to quantify uncertainty in its predictions, which will allow us to take into account uncertainty on the response. The estimated parameters $\hat{\mu}_y$ and $\hat{\sigma}_y$ will later be used to define the conditional distribution of demand within a stochastic optimization framework.

Why the loss works. In our setup, the neural network is trained to predict, for each input x_i, the parameters $\hat{\mu}_i$ and $\hat{\sigma}_i$ of a conditional normal distribution $\mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i^2)$ from which the corresponding target y_i is assumed to be drawn. This models the output as a random variable whose distribution depends on the input, being able to capture heteroscedasticity (not constant variance). Or said in another way, it is able to capture different levels of uncertainty conditioned to the input. Then, minimizing the Gaussian negative log-likelihood is finding the functions $\hat{\mu}(x)$ and $\hat{\sigma}(x)$ that maximize the likelihood that the observed data $\{(x_i, y_i)\}$, assuming that each y_i is normally distributed and comes from $y_i \sim \mathcal{N}(\hat{\mu}(x_i), \hat{\sigma}^2(x_i))$.

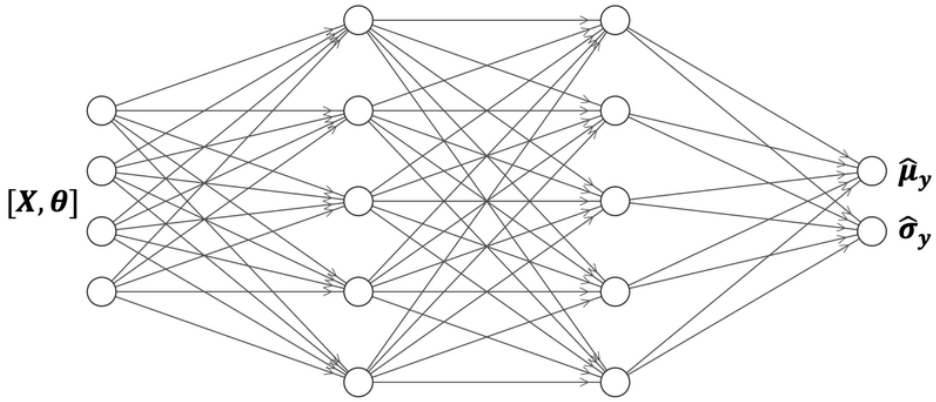


Figure 2: Structure of a Distributional Neural Network with two hidden layers. Source: [6].

Although each training pair (x_i, y_i) provides only one observed sample from its corresponding conditional distribution, the neural network generalizes across many such pairs to learn a smooth mapping from inputs to distribution parameters. The Gaussian negative log-likelihood loss encourages the predicted mean $\hat{\mu}_i$ to be close to y_i, while the predicted variance $\hat{\sigma}_i^2$ balances two competing objectives: capturing the spread of the data (error term) and avoiding unjustified overconfidence (log-variance penalty). In this way, the model learns to assign higher uncertainty to inputs with more variable responses and tighter distributions to more predictable regions.

2.2 Data generation

In order to test our model under well-controlled conditions, we generate synthetic data mimicking client-specific demand responses to various prices. Data generation is performed in the Python function `generate_data_by_ids`, whose code is given in Appendix A. The primary objective of this process is the generation of demand observations that are consistent with our modeling assumptions. Specifically, that each observation is independently and identically drawn from a normal distribution whose parameters vary across individuals and price points.

The algorithm generates data for n_{ids} clients, each of whom has two randomly drawn features, x_1 and x_2, from the uniform distribution on $[0, 1]$. For each client, we sample demand $n_{\text{prices_per_id}}$ price points randomly selected from the interval $[10, 30]$.

The demand is modeled to **decrease as price increases**, reflecting typical consumer behavior. Specifically:

- A **sigmoid function** centered at a reference price of 17 and controlled by a steepness parameter k is used to model the nonlinear decay in demand as price increases.
- The **mean demand** at each price point is calculated as a function of the characteristics of the client and the sigmoid value, appropriately scaled to ensure variation between clients.
- The **standard deviation** of demand is modeled using a Gaussian-shaped function that reaches a maximum near the center of the price range and descends toward the extremes. This reflects the idea that demand is more uncertain near the average price and more predictable when prices are clearly too low or too high. As more common is the price, there is more variability because other factors might be more important in determine the demand. If the price is too low, it will probably lead to more demand with less uncertainty, and if the price is too high, at some point no one would buy, leading to no variety around 0.

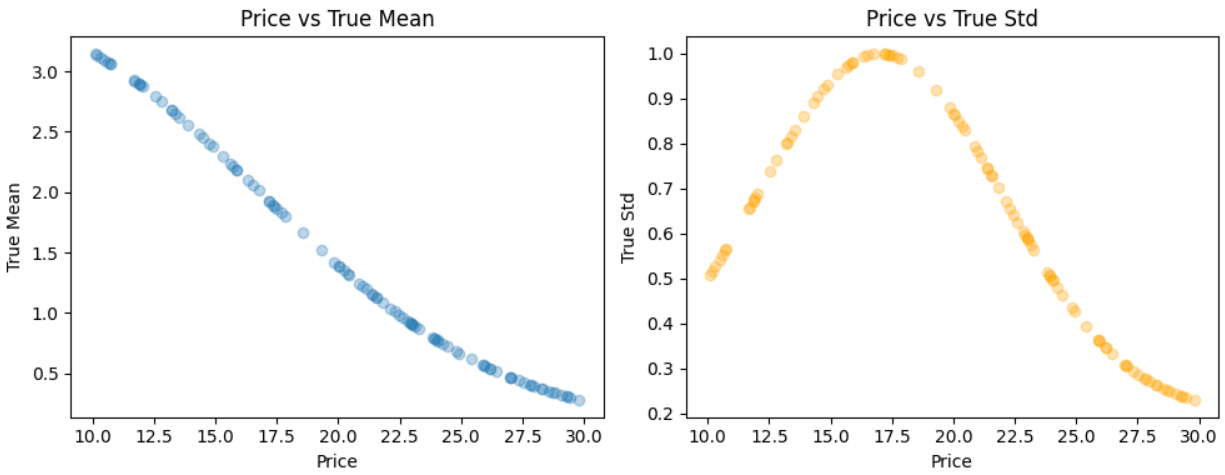


Figure 3: Illustration of true mean and std as price increases for one client

Finally, the observed demand y is sampled from a normal distribution with the computed mean and standard deviation:

$$y \sim \mathcal{N}(\mu_{\text{true}}, \sigma_{\text{true}})$$

All sampled values are truncated at zero to reflect that demand cannot be negative, as this is not contemplated on a real scenario.

This process allows us to use data that satisfy the assumption that each demand comes from a normal distribution whose parameters depend on contextual information while following a realistic behavior that could be found in real data. For this case study, 100 clients were generated, with 100 rows of simulated historical data for each one.

2.3 Model

The neural network is trained using a custom Python class called `Trainer`, which contains all the helper functions needed to split the data, train and evaluate the model, plot results, and more.

The dataset is split into training (60%), validation (20%), and test (20%) sets. In a real-world scenario, we would assume access to historical data for each client, ordered from oldest to most recent. To mimic this, the data is split in chronological order, ensuring that all clients appear in each of the three folds.

The best hyperparameters that minimize the loss are found using the Python library `hyperopt`, which performs a parallelized grid search. We fix a single hidden layer, as we aim to use the minimum number of neurons necessary. This is because the more complicated the model, the more constraints that must be included in order to integrate it into the optimization problem, resulting in longer solve times without necessarily improving predictive performance.

The hyperparameters used to train the model are the following:

- **Model configuration:**
  - Hidden layers: 1
  - Hidden size: 32

  - Dropout: 0.018
- **Training configuration:**
  - Learning rate: 0.00146
  - Weight decay: 0.00079
  - Batch size: 128
  - Max iterations: 5000
  - Early stopping patience: 500

The architecture of the neural network used is illustrated in Figure 4. It consists of a single hidden layer with 32 neurons, followed by a ReLU function and dropout. Then, the output has two branches: one for estimating the conditional mean $\hat{\mu}_y$, and one for estimating the conditional standard deviation $\hat{\sigma}_y$, with a ReLU applied to ensure non-negativity of the standard deviation. This architecture ensures fast training, preserves predictive performance, and leads to a compact and tractable embedding within the optimization model.

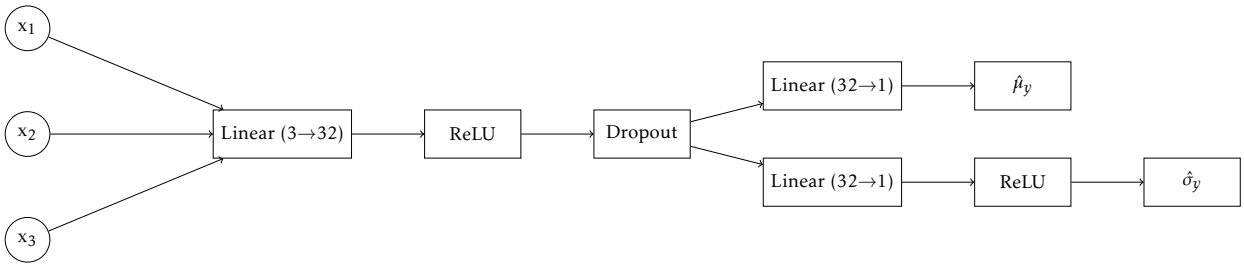


Figure 4: Architecture of the distributional neural network (DNN): one hidden layer with ReLU and dropout, followed by separate output heads for the conditional mean and standard deviation.

We assess the quality of the distributional predictions using two complementary approaches.

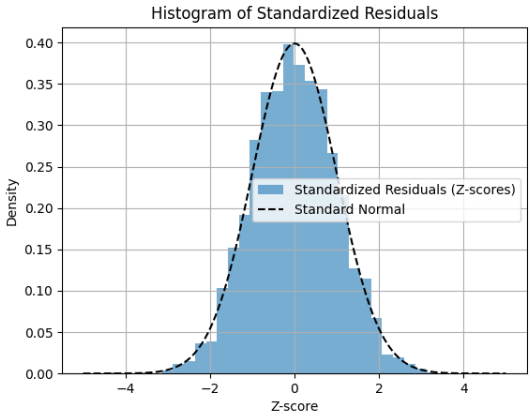
First, we evaluate the Gaussian negative log-likelihood (NLL) of the model on the validation set and compare it to a constant baseline. The baseline assumes that all outputs are drawn from a single Gaussian distribution with parameters (mean and standard deviation) estimated from the training data. Our model achieves a significantly lower Gaussian NLL loss (-0.9185) compared to the baseline (0.5103), indicating a much better fit to the data when accounting for input-dependent uncertainty. Note that the loss can take negative values, which occurs when the predicted variance becomes very small (see Equation 1). In our synthetic dataset, the predicted standard deviation is close to zero for inputs far from the center, as shown in Figure 3.

Second, we analyze the standardized residuals, computed as $z_i = (y_i - \hat{\mu}_i)/\hat{\sigma}_i$, where $\hat{\mu}_i$ and $\hat{\sigma}_i$ are the model's predicted mean and standard deviation for input x_i. If the model's variance estimates are well-calibrated, the standardized residuals should follow a standard normal distribution.

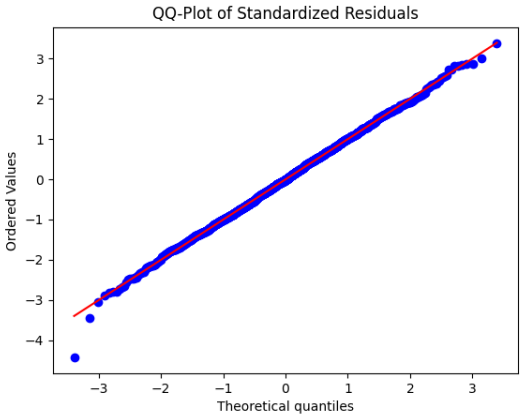
Under the model's assumption that each target follows a normal distribution $y_i \sim \mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i^2)$, the standardized residuals z_i should be distributed according to a standard normal distribution $\mathcal{N}(0,1)$. That is, if the model correctly estimates both the central tendency and the spread of the data, the transformed residuals should behave like samples from a standard Gaussian.

We verify this using a histogram, a Q–Q plot, and a Kolmogorov–Smirnov (KS) test. The KS test yields a statistic of 0.0123 and a p-value of 0.9208, indicating no significant deviation from

the standard normal distribution. This confirms that the model's predictive uncertainty estimates are statistically well-calibrated.



(a) Histogram of standardized residuals.



(b) Q–Q plot of standardized residuals.

Figure 5: Visual diagnostics for standardized residuals that shows a standard normal distribution.

3 Optimization

3.1 Optimization under Uncertainty

Uncertainty in factors like future demand, expenses, or environmental conditions frequently plays a crucial part in real-world decision-making problems. Although this assumption is rarely true in practice, classical optimization methods generally assume that all problem data are known and fixed. A framework for integrating these unknowns into the decision-making process is offered by *Optimization under uncertainty*, which results in more solid and reliable solutions.

There are numerous approaches to modeling uncertainty in optimization problems. Uncertain parameters are handled as random variables with known (or estimated) probability distributions in *stochastic programming* [7]. The objective is to maximize a cost function's expected value while taking into account limitations that might also be influenced by the uncertain parameters:

$$\min_x \ \mathbb{E}_\xi[f(x,\xi)] \quad \text{s.t.} \ \ g(x,\xi) \geq 0.$$

Sample average approximation (SAA) is a popular implementation of this strategy, in which a finite set of sampled scenarios is used to approximate the expectation.

On the other hand, *chance-constrained optimization* ensures that constraints are satisfied with high probability, which means that some constraints might sometimes be violated:

$$\min_x \ c^\top x \quad \text{s.t.} \ \mathbb{P}(g(x,\xi) \geq 0) \geq 1-\alpha.$$

Another approach is *robust optimization* [8], which will find a solution that is feasible for all values of a defined range of the uncertain variables, which is called the uncertainty set. The goal is to make decisions that perform well in the worst-case scenario:

$$\min_x \ \max_{\xi \in \mathcal{U}} f(x,\xi) \quad \text{s.t.} \ g(x,\xi) \geq 0, \ \forall \xi \in \mathcal{U}$$

Simulation-based optimization empirically assesses the quality of the solution through discrete-event simulation or Monte Carlo sampling.

$$\min_{x \in \mathcal{X}} \ \hat{f}(x) = \frac{1}{N} \sum_{i=1}^{N} f(x,\xi_i)$$

Lastly, sequential decision-making problems under uncertainty are addressed by *stochastic dynamic programming*, which is usually modeled using Markov decision processes (MDPs).

In this study, we use the stochastic programming paradigm. As explained before, a neural network is used to estimate the distribution of the uncertain demand. After that, scenarios are created using this estimated conditional distribution and incorporated into a scenario-based optimization model.

3.2 From Constraint Learning to Distributional CL

In many decision-making problems, some of the constraints involve relationships that are not explicitly known but can be inferred from historical data. As in our case, the demand y depends on the price x, but the exact functional form of this dependency is unknown. *Constraint Learning*

(CL) is a methodology that integrates machine learning models into mathematical optimization problems to capture such relationships in a data-driven manner [9, 10, 11].

Formally, given a dataset $\mathcal{D} = \{(x_i, y_i, \theta_i)\}_{i=1}^{N}$, where x represents decision variables, y is a response variable, and θ denotes contextual features, we can train a predictive model $f_{\mathcal{D}}(x, \theta)$ that approximates the mapping from (x, θ) to y. In Constraint Learning, this model is then embedded into the optimization problem by treating the prediction $y = f_{\mathcal{D}}(x, \theta)$ as a constraint. This makes it possible to optimize decisions x while accounting for their learned influence on y.

A key requirement for embedding learned models in optimization problems is that the model be *piecewise linear*. These functions are particularly attractive because they can be represented exactly using a finite number of linear constraints, possibly with additional integer (binary) variables. For example, neural networks with ReLU activation functions, decision trees, among others.

Using piecewise-linear embeddings ensures that the resulting optimization problem remains tractable and solvable by standard MIP solvers. It allows the learned constraint to behave non-linearly while still being compatible with linear optimization technology. Using a piecewise-linear activation function such as ReLU ensures that the neural network itself can still be embedded exactly using linear constraints and binary variables. If a smooth non-linear activation such as sigmoid were used instead, the resulting embedding would require general non-convex nonlinear programming techniques, making the overall problem significantly harder to solve in practice.

Importantly, Constraint Learning differs from standard predictive modeling because the learned model is explicitly used to define a constraint that involves the decision variables. That is, the optimization does not simply take the prediction as an external input, but it reasons about how different decisions x will influence the learned output y, and uses that reasoning to make optimal choices. This is especially useful when the response variable (demand) is directly influenced by decisions (decided price).

As explained in Section 2.1.2, we are not aiming for point prediction, that is what the models traditionally embedded in Constraint Learning do. To avoid solutions that might be overconfident and suboptimal in practice, we use *Distributional Constraint Learning* (DCL), which extends the CL framework by using models that take into account the uncertainty by modeling the distribution of the response [6]. This allows the optimization model to reason not just about the expected value of y, but about its variability and risk profile under different decisions x.

Our distributional neural network (DNN) is trained to model each y_i as independently drawn from a normal distribution conditioned on its input, and the piecewise-linear structure allows the embedding into the optimization problem using mixed-integer linear constraints.

Once this distribution is estimated ($\hat{\mu}_i$ and $\hat{\sigma}_i$), it can be used to generate scenarios (sample realizations of y_i) within the optimization model, enabling stochastic programming techniques such as Sample Average Approximation (SAA).

3.3 Dynamic pricing problem

We begin by introducing the general formulation of the Distributional Constraint Learning (DCL) methodology for optimization under uncertainty. In this framework, variables x are chosen, while the response variable y is modeled as a random variable whose distribution depends on x

and contextual features θ. The goal is to minimize the expected value of a cost function that depends on both x and the random variable $\tilde{y}$:

$$\min_{x,\tilde{y}} \quad \mathbb{E}_{\tilde{y}\sim\mathcal{Y}(\hat{\mu},\hat{\sigma})}[c(x,\tilde{y};\theta,\xi)] \tag{1a}$$

$$\text{s.t.} \quad g(x,\tilde{y}) \leq 0 \tag{1b}$$

$$\hat{\mu},\hat{\sigma} = f_D(x;\theta,\xi) \tag{1c}$$

Here, $\hat{\mu}$ and $\hat{\sigma}$ are the predicted parameters of the conditional distribution of y, obtained from a piecewise-linearizable model f_D trained on historical data. The variable ξ captures additional exogenous uncertainty.

To solve the problem in practice, we use a Sample Average Approximation (SAA), which replaces the expectation with an empirical average over a finite set of sampled scenarios $\omega \in \Omega$. Each scenario is generated using the estimated mean and standard deviation:

$$\min_{x,y_\omega} \quad \sum_{\omega\in\Omega} \pi_\omega c(x,y_\omega;\theta,\xi) \tag{2a}$$

$$\text{s.t.} \quad g(x,y_\omega) \leq 0, \quad \forall\omega\in\Omega \tag{2b}$$

$$\hat{\mu}_y,\hat{\sigma}_y = f_D(x;\theta,\xi) \tag{2c}$$

$$y_\omega = z_\omega\cdot\hat{\sigma}+\hat{\mu}, \quad \forall\omega\in\Omega \tag{2d}$$

Where $\pi_\omega = \frac{1}{|\Omega|}$ is the probability of each scenario. Each z_ω is a sample from a standard normal distribution, and y_ω represents the corresponding realization of the random response variable under scenario ω.

We now adapt this general framework to our specific case study, where the objective is to determine the optimal prices of a product for a set of clients in order to maximize expected revenue under demand uncertainty.

3.3.1 Notation

Let $\mathcal{N}$ be the set of bars (indexed by n), and Ω the set of scenarios (indexed by ω), with $|\Omega| = n_{\text{sc}}$ and $\pi_\omega = \frac{1}{|\Omega|}$ the scenario probabilities.

- $x_{\text{price},n}$: decision variable representing the price offered to client n.
- θ_n: contextual (fixed) features for client n.
- $\hat{\mu}_n$, $\hat{\sigma}_n$: predicted mean and standard deviation of demand for bar n.
- $y_{n,\omega}$: stochastic realization of demand for bar n under scenario ω (continuous).
- $y^{\text{int}}_{n,\omega}$: integer-valued approximation of demand, if required.
- r_ω: revenue in scenario ω.
- z_ω: Random sample for scenario ω coming from $z\sim\mathcal{N}(0,1)$
- $\bar{x}_n$: historical average price offered to bar n.
- $P_{\max}$: upper bound for any price.

3.3.2 Conceptual Model

The goal is to optimize prices $x_{\text{price},n}$ to maximize expected revenue across all bars and scenarios, accounting for the uncertainty in demand. The demand distribution parameters are learned through a neural network that takes both decision variables and contextual information as input. The model is defined as follows:

$$\max_{x,y,r} \quad \sum_{\omega \in \Omega} \pi_\omega \cdot r_\omega \tag{P.1}$$

$$\text{s.t.} \quad r_\omega = \sum_{n \in \mathcal{N}} x_{\text{price},n} \cdot y^{\text{int}}_{n,\omega}, \quad \forall \omega \in \Omega \tag{P.2}$$

$$y_{n,\omega} = \hat{\mu}_n + z_\omega \cdot \hat{\sigma}_n, \quad \forall n, \omega \tag{P.3}$$

$$y^{\text{int}}_{n,\omega} \in [y_{n,\omega} - 0.5, y_{n,\omega} + 0.5], \quad \forall n, \omega \tag{P.4}$$

$$\hat{\mu}_n, \hat{\sigma}_n = f_{\mathcal{D}}(x_{\cdot,n}), \quad \forall n \tag{P.5}$$

$$\frac{1}{|\mathcal{N}|} \sum_{n \in \mathcal{N}} x_{\text{price},n} \geq \bar{x} \cdot 1.02 \tag{P.6}$$

$$x_{\text{price},n} \leq \bar{x}_n \cdot 1.05, \quad \forall n \tag{P.7}$$

$$0 \leq x_{\text{price},n} \leq P_{\max}, \quad \forall n \tag{P.8}$$

$$\text{Contextual variables fixed: } x_{k,n} = \theta_{n,k}, \quad \forall k \neq \text{price} \tag{P.9}$$

Here, the neural network $f_{\mathcal{D}}$ is embedded into the model using piecewise-linear constraints based on ReLU activations, as described in Section 3.3.3.

- **(P.1):** Weighted average of the revenues of the scenarios.
- **(P.2):** The revenue r in scenario ω is defined as the sum of all the prices of the clients for that scenario multiplied by the integer demand $y^{\text{int}}_{n,\omega}$.
- **(P.3):** Realization of the demand for the client n and scenario ω. Calculated using a sample from $\mathcal{N}(0,1)$, scaling using the standard deviation, and shifted by the mean. The ones used are $\hat{\mu}_n$, $\hat{\sigma}_n$, the ones predicted by the model for that client. This is equivalent to sample from $\mathcal{N}(\hat{\mu}_n, \hat{\sigma}_n)$
- **(P.4):** Rounding constraints of $y_{n,\omega}$ to the closest integer value. This is performed using 2 constraints. One limits the upper bound, and one limits the lower bound. Only one integer value should be in the range $[y_{n,\omega} - 0.5, y_{n,\omega} + 0.5]$ and is the corresponding to $round(y_{n,\omega})$. It is necessary as products are measured in units, and decimals would make no sense for this case study. The $y_{n,\omega}$ are continuous, as they are samples from a normal distribution.
- **(P.5):** Mean and standard deviation computed by the neural network for the variable price $x_{\text{price},n}$ and the fixed contextual information $x_{k,n} \forall k \neq$ price.
- **(P.6):** Force the average price offered to be at least 2% higher than the historical mean.
- **(P.7):** Price offered cannot be higher than 5% more than the historical mean of that client. It is necessary to set a tight upper limit, as to fulfill $(P.6)$, the optimal solution could be setting for one or a few clients a really high price. That would make those clients to churn due to the drastic increase in the price offered and a demand of 0.
- **(P.8):** General lower and upper bound. The lower bound is 0, as negative prices should not be allowed. An upper limit is also important to not explore solutions that make no practical sense. Limiting the variable of interest to a small range, makes the convergence to a solution faster. Also, as the demand on the training data is also limited in range, the solutions that go beyond the limits might not be reliable. For example, if setting a really high price leads

for some reason to a demand of 1, that would be the solution that maximizes ($P.2$), even if makes no sense.

- **(P.9)**: Fixing contextual variables of the client that are not the price. The price must be a free variable, as it is what we are interested in optimize. But the other variables should remain stable, as are required by ($P.5$) to predict $\hat{\mu}_n, \hat{\sigma}_n$.

The function used to define the conceptual model can be seen in Appendix B

3.3.3 Neural Network Embedding via Piecewise-Linear Constraints

The structure of the neural network follows a standard feedforward architecture with L layers: an input layer, $L-2$ hidden layers, but as mentioned before, in our case two output layers. Each hidden node applies a ReLU activation function to a linear combination of the outputs from the previous layer.

A hidden layer l has N^l nodes. Let v_i^l denote the value of node $i \in N^l$, and let $\tilde{v}_i^l$ be the pre-activation value, computed as:

$$\tilde{v}_i^l = b_i^l + \sum_{j \in N^{l-1}} w_{ij}^l v_j^{l-1} \tag{NN.1}$$

where w_{ij}^l are the weights and b_i^l the bias term. The activated output is given by:

$$v_i^l = \max\{0, \tilde{v}_i^l\}$$

Several works, such as [2], study how to embed trained neural networks with ReLU activation functions as mixed-integer formulations. That work also presents an improvement over the standard big-M formulation by using two bounds instead of one. Specifically, using M_{up} and M_{low} improves the formulation by tightening the feasible region and helping solvers converge faster and more reliably:

$$v_i^l \geq \tilde{v}_i^l \tag{NN.2}$$
$$v_i^l \leq \tilde{v}_i^l - M_{\text{low}}(1 - j_i^l) \tag{NN.3}$$
$$v_i^l \leq M_{\text{up}} j_i^l \tag{NN.4}$$
$$j_i^l \in \{0, 1\} \tag{NN.5}$$

Here, j_i^l is a binary variable indicating whether node i in layer l is active ($\tilde{v}_i^l > 0$), and $M_{\text{low}}, M_{\text{up}}$ are lower and upper bounds on $\tilde{v}_i^l$, respectively.

The linear combination of weights in Equation (NN.1), together with the linearized ReLU formulation in Equations (NN.2)–(NN.5), enables the full embedding of the DNN structure in the optimization problem.

The input layer takes as input both the decision variables $\mathbf{x}$ and the contextual features $\boldsymbol{\theta}$, which are standardized using training set statistics. These values propagate through the hidden layers according to the rules above. Predictions for $\hat{\mu}_y$ and $\hat{\sigma}_y$, which specify the conditional distribution of the learned variable y, are generated by the output layer. These predicted parameters allow for scenario generation and the embedding of uncertainty directly into the optimization model.

The function used to define the conceptual model can be seen in Appendix C

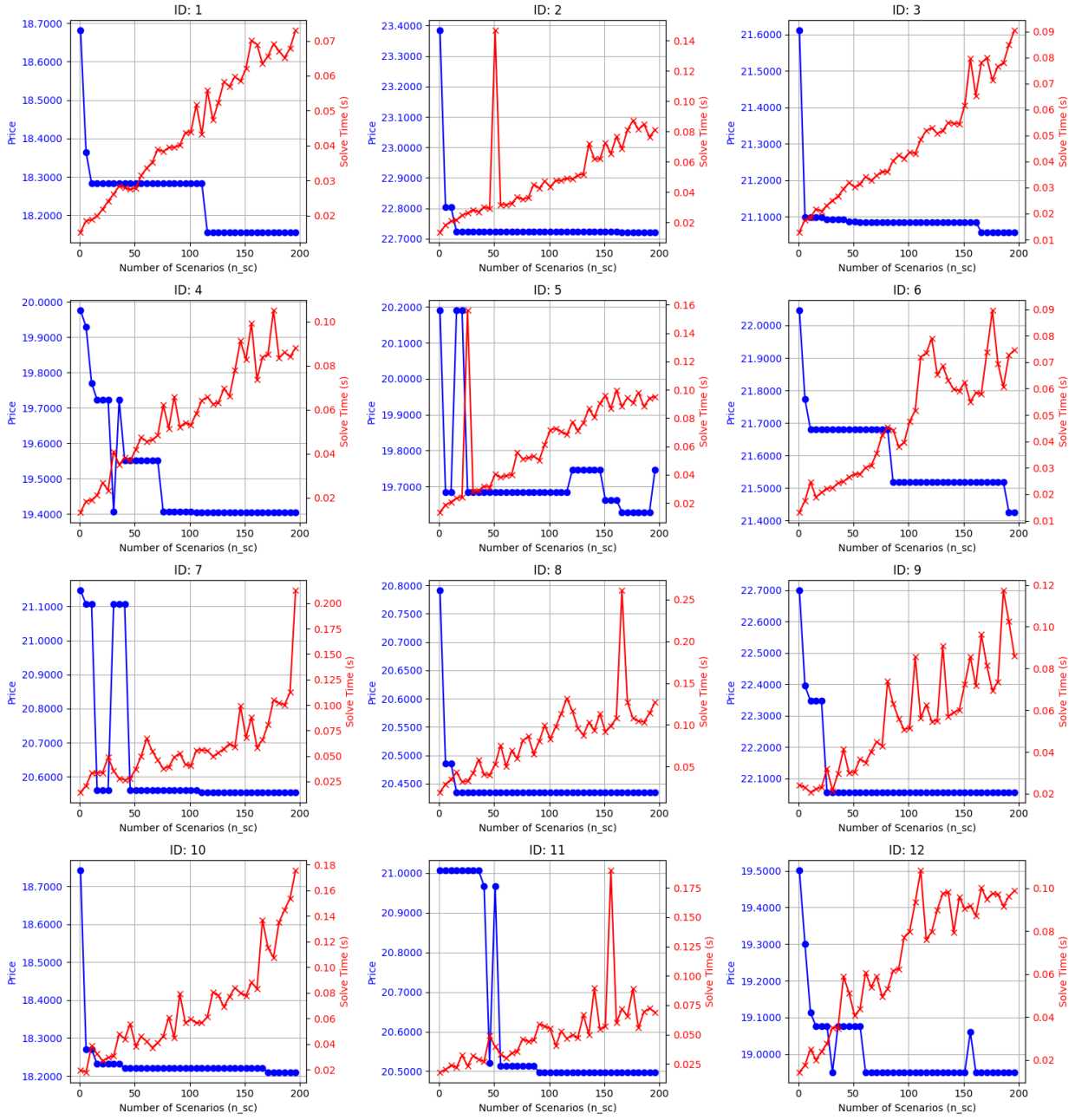
3.4 Selection of number of scenarios

The selection of the number of scenarios is a crucial step. The more scenarios, the more decision variables and constraints required $(P.2), (P.3), (P.4)$. Again we aim to keep the model as simple as possible while keeping the good performance. Then, the optimal number of scenarios, is the one that provides a stable pricing with the minimum number of constraints.

We solve this problem empirically, selecting a few random ids, and solve the problem for increasing number of scenarios for each one individually. Then, analyzing the plotted price and execution time, we choose the number of scenarios that provides a solid pricing.

The optimization problem is solved with and without the constraints that take into account the global average $(P.6)$ and the client average $(P.7)$. We will call those constraints as the *mean constraints* from now on, as they limit the price based on means. Using them, make the prices more stable, as the feasible region is smaller. As it is solved client by client, the global average $\bar{x}$ and the client average $\bar{x}_n$ are the same, leading to a constraints that requires the price to be between 2% and 5% above the average, that is, within the range $[\bar{x}_n \times 1.02,\ \bar{x}_n \times 1.05]$.

Let's start analyzing the number of scenarios when using the *mean constraints*:

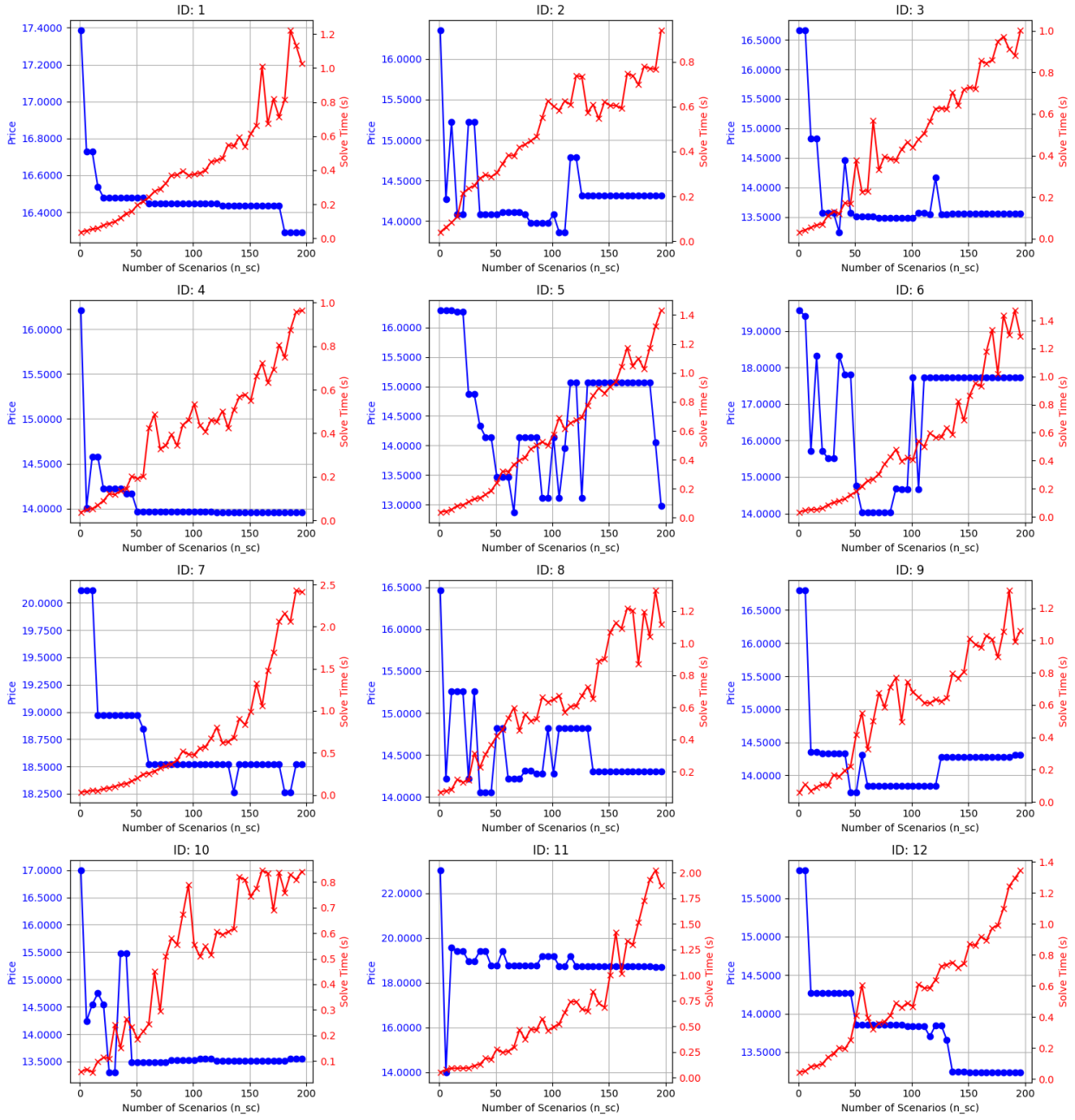


(a) Number of scenarios vs price.

Figure 6: Visual diagnostics for choosing the number of scenarios. Using constraints ($P.6$) and ($P.7$)

In the image we performed the experiment in 12 random clients. The higher variability at the left of the x axis is a common pattern, indicating that with very few scenarios the price is less reliable. As the number of scenarios increases, in most cases the price gets fairly stable. Even in some cases it might look unstable, it is important to take a look on the scale of the price axis, as the prices are only differing mainly on the decimals, being not a really important difference in our case.

The following plots correspond to the same experiment without using the *mean constraints*.



(a) Number of scenarios vs price.

Figure 7: Visual diagnostics for choosing the number of scenarios. Without using constraints ($P.6$) and ($P.7$)

In general, we observe the same pattern, but in this case with more overall variability. This is because without the strong limitations on price per client, the range of feasible prices is wider. We can observe this on the price axis.

In both cases, a step function behavior is usual. This is because we use the integer approximation of the demand, creating also steps on the price. A good example of this is the following plot, created by running the experiment on client 2 without the *mean constraints* and using the continuous demand.

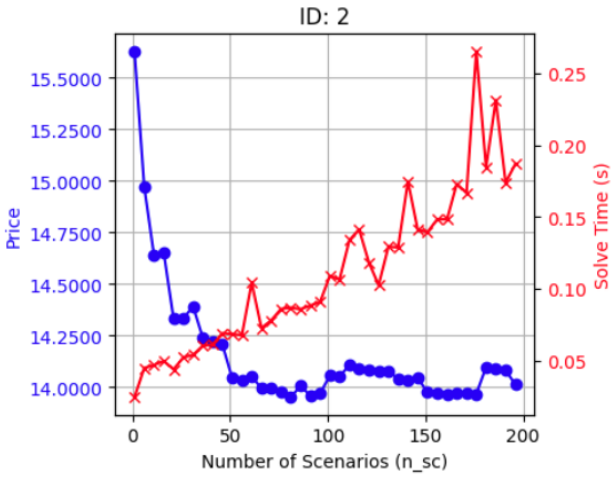


Figure 8: Visual diagnostics for choosing the number of scenarios on client 2. Without using constraints ($P.6$) and ($P.7$), and with continuous demand.

In conclusion, we consider 50 scenarios enough to stabilize the prices.

3.5 One client vs N clients

The optimization process can be solved almost instantly for a single client, as shown on the right y-axis of the plots in Figure 7. If we wanted to solve the problem for n clients, we could solve it individually for each one, and the total solution time would scale linearly with the number of clients.

In our case, constraint ($P.6$) links all clients, so solving the problem for only one in isolation would be meaningless. This makes scalability with respect to the number of clients a challenge, since each additional client introduces new variables and constraints, increasing the overall complexity. The table below shows how the number of constraints grows with the number of clients.

Number of clients	Number of constraints
1	304
2	557
3	810
4	1063
5	1316

Table 1: Number of constraints as a function of the number of clients.

As we can see, each additional client adds 253 constraints to the problem.

We solve the problem iteratively by increasing the number of clients and recording the execution time in order to assess how the number of clients affects execution time. Additionally, we examine the effect of using the *mean constraints*. While these constraints increase the total number of constraints, they significantly reduce the feasible region, improving the solver's convergence and enabling the problem to be solved for more clients in a reasonable amount of time.

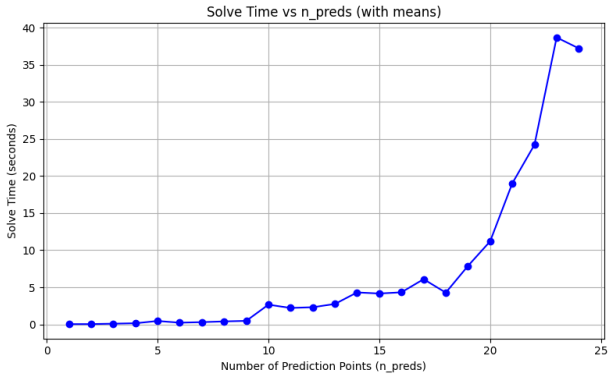


Figure 9: Execution time vs. number of clients using *mean constraints*.

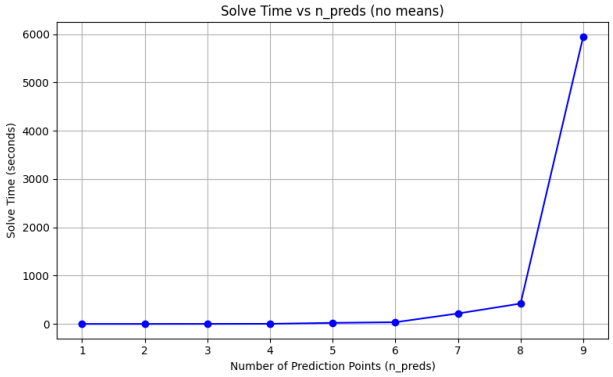


Figure 10: Execution time vs. number of clients without using *mean constraints*.

In both cases, we observe a clear exponential trend. Although the number of clients that can be solved in a reasonable time increases when using the *mean constraints*, a point is reached where the problem becomes intractable. This highlights one of the main limitations of this methodology.

To address this issue in future work, decomposition techniques could be explored. These methods are designed to break down large optimization problems with complicating constraints, such as $(P.6)$ into smaller, more manageable subproblems that can be solved iteratively or in parallel [12].

3.6 Solution for 5 clients

In this section, we solve the problem for 5 clients and analyze the solution. At this point, the neural network is already trained, so after defining the model as explained in Section 3.3.2, the neural network is embedded as described in Section 3.3.3. Both implementations are heavily inspired by [13].

Before solving the problem, we need to retrieve the contextual information of the 5 clients in the test set.

Id	x1	x2
1	0.3745401188	0.0314291857
2	0.9507143064	0.6364104113
3	0.7319939418	0.3143559811
4	0.5986584842	0.5085706912
5	0.1560186404	0.9075664739

Table 2: Contextual information for 5 clients

We also need the global average price among these clients in the training set (used for constraint ($P.6$)), and the average price for each client in the training set (used for constraint ($P.7$)).

Id	Mean
1	20.23
2	19.98
3	19.08
4	19.01
5	21.34

Table 3: Average price per client in the training set.

Global mean: 19.93

Once `Gurobi` finds the optimal solution, we can analyze the results.

Id	Average demand	Average demand int	Optimal price	Demand $\hat{\mu}_n$	Demand $\hat{\sigma}_n$
1	1.24	1.18	21.24	1.14	0.67
2	5.88	5.92	18.27	5.76	0.85
3	3.50	3.54	19.78	3.38	0.82
4	3.43	3.46	19.93	3.31	0.83
5	1.86	1.9	22.41	1.76	0.67

Table 4: Results of optimization problem

The maximum value of the objective function ($P.1$) is 314.78. This can be verified by computing (Average demand int)$^T \cdot$ Optimal price = $1.18 \cdot 21.24 + 5.92 \cdot 18.27 + 3.54 \cdot 19.78 + 3.46 \cdot 19.93 + 1.90 \cdot 22.41 = 314.78$. The average optimal price is 20.32, which satisfies the constraint of being greater than or equal to $19.93 \cdot 1.02$. Additionally, none of the prices per client exceeds 5% over their historical average.

Note that each of the 50 scenarios yields a demand value. The average of these demands appears in the *Average demand* column. The *Average demand int* column shows the average of the rounded demands for each scenario.

We can visualize the distribution of the demand. First, we show the overall demand distribution for the 5 clients, comparing continuous and integer demand values.

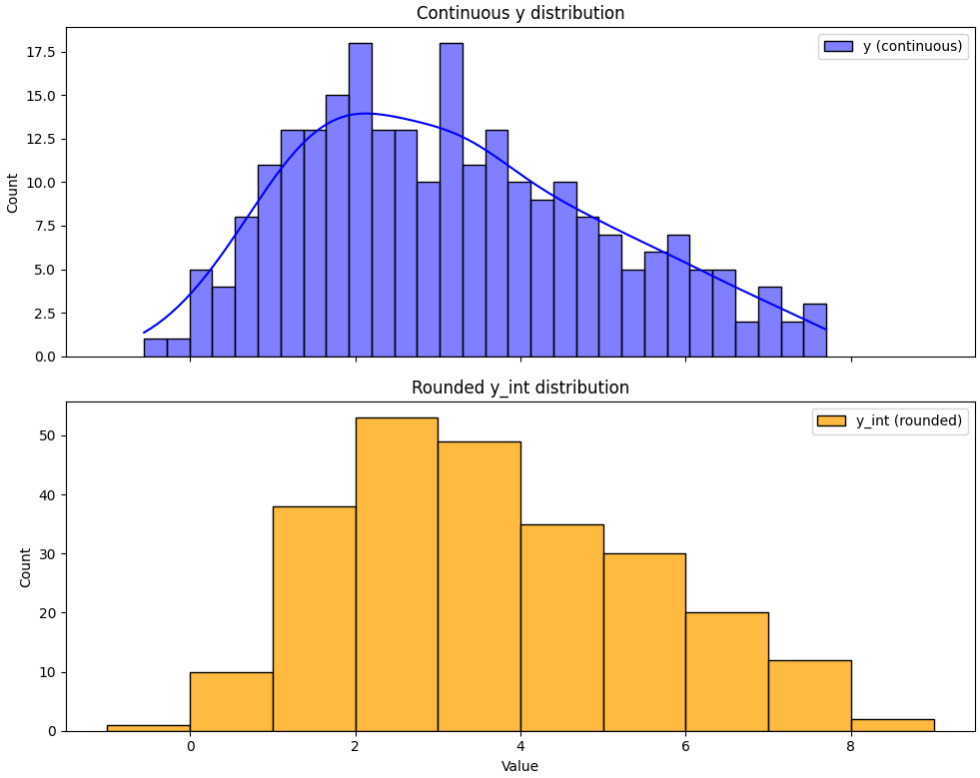


Figure 11: Overall distribution of the demand for 5 clients.

A more informative view is the demand distribution broken down by client, as each one follows a different distribution.

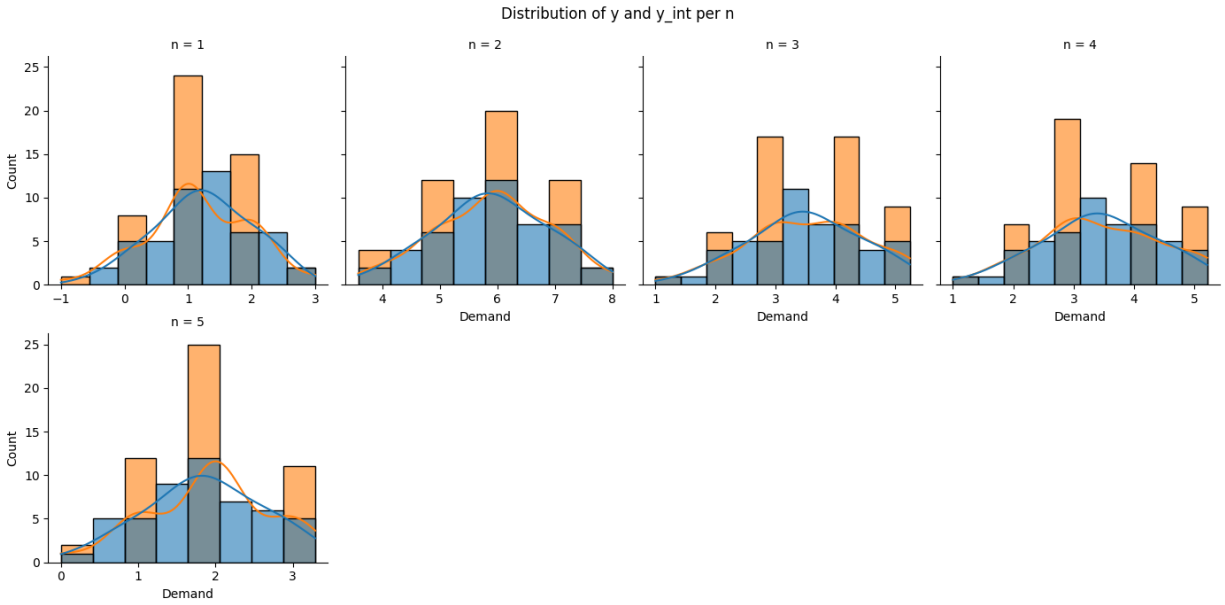


Figure 12: Distribution of the demand by client.

We observe that, for client 1, some scenarios generated negative demand values. Recall that we assume the demand follows a normal distribution, so under certain conditions—such as a low $\hat{\mu}_n$, high $\hat{\sigma}_n$, or both, it is possible to observe negative realizations. Nonetheless, the average demand remains positive, and since the training data contains no negative values, the neural network has successfully learned to model the distribution.

4 Conclusion

In conclusion, Distributional Constraint Learning, which was already proved to work in other settings like energy price predictions, can be adapted to other industry situations as dynamic pricing.

As we have seen in this case study, as long as the basic assumptions are met, the Distributional Neural Network can effectively learn the conditional distribution of the response. This network can be embedded as linear constraints if the appropriate structure is used, and after formulating correctly the problem to solve, the solutions found in a reasonable time are satisfactory. The solution also allows us to get an approximation of the demand distribution from where the different scenarios sampled the demand prediction, allowing us to see how much information we would lose if we were just using the conditional mean in point prediction.

Some limitations arose during the experimentation, like the bad scaling as the number of clients increases when using constraints that force all the clients to be solved simultaneously. The next steps would certainly involve exploring decomposition techniques to tackle this problem, but we consider those to be out of the scope of this case study.

A Appendix: Synthetic Data Generation Code

```python
def generate_data_by_ids(n_ids=100, n_prices_per_id=10, seed=42, k=0.35):
    np.random.seed(seed)

    # Create one (x1, x2) pair per ID
    x1_vals = np.random.uniform(0, 1, size=n_ids)
    x2_vals = np.random.uniform(0, 1, size=n_ids)
    ids = np.arange(1, n_ids + 1)

    # Create all combinations of ID and price
    data = []
    for i in range(n_ids):
        id_ = ids[i]
        x1 = x1_vals[i]
        x2 = x2_vals[i]

        # Generate n_prices_per_id random prices between 10 and 30
        prices = np.random.uniform(10, 30, size=n_prices_per_id)

        for price in prices:
            # Demand decay using sigmoid
            center = 17
            sigmoid = 1 / (1 + np.exp(k * (price - center)))

            # Base demand, scaled up
            raw_demand = (5 * x1 + 3 * x2)
            base_demand = raw_demand * sigmoid * 2  # scaling factor

            # Mean
            true_mean = base_demand if 10 <= price <= 30 else 0

            # Std: bell-shaped curve, low at extremes
            std_peak = 1.0
            std_min = 0.2
            width = 5
            base_std = std_min + (std_peak - std_min) * np.exp(-((price -
                center) ** 2) / (2 * width ** 2))
            true_std = base_std if 10 <= price <= 30 else 0

            # Sample demand
            y = np.random.normal(loc=true_mean, scale=true_std)
            y = max(0, y)

            data.append({
                'id': id_,
                'x1': x1,
                'x2': x2,
                'price': price,
                'true_mean': true_mean,
                'true_std': true_std,
                'y': y
            })

    df = pd.DataFrame(data)
    return df
```

Listing 1: Python function used to generate synthetic data

B Appendix: Conceptual model definition code

```
def model_definition(contextual_var, n_sc, n_preds, integer_demand=False,
    mean_price=None, mean_price_by_id=None, P_max=30):
    model = pyo.ConcreteModel()

    model.N = pyo.RangeSet(0,n_preds-1)
    model.W = pyo.RangeSet(1,n_sc) # scenarios

    if mean_price_by_id is not None:
        model.mean_price_by_id = pyo.Param(model.N, initialize=
            mean_price_by_id)


    pi = [1/n_sc for i in range(n_sc)]

    model.var_ind = pyo.Set(initialize=contextual_var.columns.sort_values())

    model.x = pyo.Var(model.var_ind, model.N) # price

    model.y = pyo.Var(pyo.Any, dense=False, domain=pyo.Reals)
    if integer_demand:
        model.y_int = pyo.Var(model.N, model.W, domain=pyo.Integers)

    model.rent = pyo.Var(model.W, within=pyo.Reals) # rentabilidad per
        scenario

    #definition of the objective function
    def obj_expression(model):
        return sum(pi[w-1] * model.rent[w] for w in model.W)
    model.OBJ = pyo.Objective(rule=obj_expression, sense=pyo.maximize)

    ##################
    ## Constraints ##
    ##################

    if integer_demand:
        # rentability constraint integer y
        def rent_expression_int(model, w):
            return model.rent[w] == sum(model.y_int[n, w]*model.x['price',n]
                for n in model.N)
        model.rent_expression_int = pyo.Constraint(model.W, rule=
            rent_expression_int)

        # integer demand constraint
        # Upper bound constraint
        def round_y_upper(model, n, w):
            return model.y_int[n, w] <= model.y['demand', n, w] + 0.5
        model.round_y_upper = pyo.Constraint(model.N, model.W, rule=
            round_y_upper)

        # Lower bound constraint
        def round_y_lower(model, n, w):
            return model.y_int[n, w] >= model.y['demand', n, w] - 0.5
        model.round_y_lower = pyo.Constraint(model.N, model.W, rule=
            round_y_lower)
    else:
        # rentability constraint continuous y
```

```
        def rent_expression(model, w):
            return model.rent[w] == sum(model.y['demand', n, w]*model.x['price
                ',n] for n in model.N)
        model.rent_expression = pyo.Constraint(model.W, rule=rent_expression)


    #constraint for fixing contextual information
    def contextual_info(model,x_ind,n):
        if x_ind=='price':
            return pyo.Constraint.Skip
        else:
            return model.x[x_ind,n] == contextual_var.loc[n,x_ind]
    model.const9 = pyo.Constraint(model.var_ind, model.N, rule=contextual_info
        )

    # max price constraint for each price
    def max_price_expression(model, n):
        return model.x['price', n] <= P_max

    model.max_price_expression = pyo.Constraint(model.N, rule=
        max_price_expression)

    def min_price_expression(model, n):
        return model.x['price', n] >= 0
    model.min_price_expression = pyo.Constraint(model.N, rule=
        min_price_expression)

    if mean_price is not None:
        # price increase of 2% with respect the mean price
        def price_increase_expression(model):
            return sum(model.x['price', n] for n in model.N)/len(model.N) >=
                mean_price * 1.02
        model.price_increase_expression = pyo.Constraint(rule=
            price_increase_expression)

        if mean_price_by_id is not None:
            # price can't increase more than 5% with respect the mean price
                offered to this client in the past
            def price_increase_by_id_expression(model, n):
                return model.x['price', n] <= model.mean_price_by_id[n] * 1.05

            model.price_increase_by_id_expression = pyo.Constraint(model.N,
                rule=price_increase_by_id_expression)

    return model
```

Listing 2: Python function used to define the conceptual model

C Appendix: Distributional Neural Network Embedding code

```
def const_embed(model, opt_model, constaints, outcome, n_scenarios = 1,
    n_preds=1, deterministic = False):
        '''
        This function embdeds the fitted prediction model within the
            optimization problem.
```

```
        Expecting a defined optimization model "opt_model", constraint
            dataframe "constraints",
        a name for an "outcome", and the number of scenarios to generate.
        '''

        M_l=-1e3
        M_u=1e3
        opt_model.v = Var(Any, dense=False, domain=NonNegativeReals)
        opt_model.v_ind = Var(Any, dense=False, domain=Binary)

        for n in range(n_preds):
            max_layer = max(constaints['layer'])
            nodes_input = range(len(model.X_train.columns))
            v_input = [(opt_model.x[name,n]-model.X_mean[i])/model.X_std[i]
                for i,name in enumerate(model.X_train.columns)] # If data is
                not scaled
            #v_input = [opt_model.x[name, n] for name in model.X_train.columns
                ] # If data is scaled

            for l in range(max_layer):
                df_layer = constaints.query('layer == %d' % l)
                max_nodes = [k for k in df_layer.columns if 'node_' in k]
                # coeffs_layer = np.array(df_layer.iloc[:, range(len(max_nodes
                    ))].dropna(axis=1))
                coeffs_layer = np.array(df_layer.loc[:, max_nodes].dropna(axis
                    =1))
                intercepts_layer = np.array(df_layer['intercept'])
                nodes = df_layer['node']

                if l == max_layer-1:
                    node = nodes.iloc[0]
                    opt_model.add_component('Mean_est' + str(n), Constraint(
                        rule=opt_model.y[outcome, n,'mean'] == (sum(v_input[i]
                            * coeffs_layer[node, i] for i in nodes_input) +
                        intercepts_layer[node])))

                    l = l+1
                    df_layer = constaints.query('layer == %d' % l)
                    max_nodes = [k for k in df_layer.columns if 'node_' in k]
                    coeffs_layer = np.array(df_layer.loc[:, max_nodes].dropna(
                        axis=1))
                    intercepts_layer = np.array(df_layer['intercept'])
                    opt_model.add_component('Var_est' + str(n), Constraint(
                        rule=opt_model.y[outcome, n,'sd'] == (sum(v_input[i] *
                            coeffs_layer[node, i] for i in nodes_input) +
                        intercepts_layer[node])))

                else:
                    # Save v_pos for input to next layer
                    v_pos_list = []
                    for node in nodes:
                        ## Initialize variables
                        v_pos_list.append(opt_model.v[(outcome, l, node, n)])
                        opt_model.add_component('constraint_1_layer' + str(l)
                            + '_node' + str(node)+ '_pred'+ str(n)+ outcome,
                                        Constraint(rule=opt_model.v[(
                                            outcome, l, node, n)] >= sum(
                                            v_input[i] * coeffs_layer[node
```

```
                                        , i] for i in nodes_input) +
                                        intercepts_layer[node]))
                    opt_model.add_component('constraint_2_layer' + str(l)
                        + '_node' + str(node)+ '_pred'+ str(n)+ outcome,
                                    Constraint(rule=opt_model.v[(
                                        outcome, l, node, n)] <= M_u *
                                         (opt_model.v_ind[(outcome, l,
                                         node, n)])))
                    opt_model.add_component('constraint_3_layer' + str(l)
                        + '_node' + str(node)+ '_pred'+ str(n)+ outcome,
                                    Constraint(rule=opt_model.v[(
                                        outcome, l, node, n)] <= sum(
                                        v_input[i] * coeffs_layer[node
                                        , i] for i in nodes_input) +
                                        intercepts_layer[node] - M_l *
                                         (1 - opt_model.v_ind[(outcome
                                        , l, node, n)])))
                ## Prepare nodes_input for next layer
                nodes_input = nodes
                v_input = v_pos_list


    if deterministic == False:
        np.random.seed(0)
        z_vals = np.random.normal(size=n_scenarios)

        for n in range(n_preds):
            for w in range(1, n_scenarios + 1):
                # opt_model.y[outcome, n,'sd'] and opt_model.y[outcome, n
                    ,'mean'] are already defined in the model as learned
                    variables above. Samples from standard normal
                    distribution are added to the mean and sd of the
                    prediction.
                # The mean and sd are scaled to the original scale of the
                    data using self.y_std and self.y_mean.
                opt_model.add_component(outcome + '_pred' + str(n) + '_sce
                    ' + str(w), Constraint(rule=opt_model.y[outcome, n, w]
                     == (z_vals[w-1] * opt_model.y[outcome, n,'sd'] +
                    opt_model.y[outcome, n,'mean'])*model.y_std + model.
                    y_mean))
    else:
        for n in range(n_preds):
            for w in range(1, n_scenarios + 1):
                opt_model.add_component(outcome + '_pred' + str(n) + '_sce
                    ' + str(w), Constraint(rule=opt_model.y[outcome, n, w]
                     == opt_model.y[outcome, n,'mean']*model.y_std + model
                    .y_mean))
```

Listing 3: Python function used to embed the neural network as Piecewise-Linear Constraints

D Appendix: Time Distribution and Development Timeline

In this section, I will detail as much as I can the development time of the project. To be honest, I did not track the hours used in total, much less the time per section. So as providing hours would be really inaccurate, I will specify the parts of the process, during what time I was working on each one, and an approximated percentage of the total time for them.

D.1 Development Timeline

The following timeline reflects the approximate chronological order in which the main components of the work were developed. Everything was done in the academic year 2024-2025. Some parts overlap as they were developed simultaneously.

1. **November – January**: Start drafting the project.
2. **February – March**: Understand the theoretical concepts and study prior work.
3. **March – April**: Analyze the reference implementation [13] and formulate the case study.
4. **March – May**: Design and develop the components described in the *Demand Prediction Model* section.
5. **May – June**: Formulate and solve the optimization problem. What is described in the *Optimization* section.
6. **June**: Write and conduct the final review of the thesis document.

D.2 Estimated Time Distribution

The table below summarizes the percentage of total time dedicated to each of the previously defined components.

Component	Estimated Time (%)
1	5%
2	10%
3	5%
4	30%
5	30%
6	20%
Total	**100%**

E Appendix: Declaration of use of Generative AI in the Master's Thesis

I have used Generative AI in this work?

Mark as appropriate:

■ YES
☐ NO

If you checked YES, complete the following 3 parts:

Part 1: Reflection on Ethical and Responsible Behavior

Answer the following questions honestly:

1. In my interaction with Generative AI tools, I have submitted sensitive data with proper authorization:
   - ☐ YES, with authorization
   - ☐ NO, without authorization
   - ■ NO, I have not used sensitive data
2. I have submitted materials protected by copyright with proper authorization:
   - ☐ YES, with authorization
   - ☐ NO, without authorization
   - ■ NO, I have not used protected materials
3. I have submitted personal data with proper authorization:
   - ☐ YES, with authorization
   - ☐ NO, without authorization
   - ■ NO, I have not used personal data
4. I have respected the terms of use of the AI tool and ethical principles:
   - ■ YES
   - ☐ NO

Part 2: Technical Use Declaration

I declare having used the Generative AI system (*ChatGPT/Github Copilot*) for:

Documentation and writing

- Support for reflection:
  Discussing theoretical or practical doubts in dialogue format with ChatGPT.
  Learning about common practices in certain fields.
- Review or rewriting of paragraphs:
  Rewriting my own texts while keeping the explanation order, but correcting grammatical and syntactical mistakes.
- Search for information or specific answers.
  Searching for information on how to use certain libraries.

Development of specific content

- Assistance in code writing (programming).
  Requesting basic code structures I have already implemented many times to avoid starting from scratch.
- Generation of diagrams, images, audio, or videos.
  Help with the creation of graphics, in Python and in LaTeX.
- Optimization processes.
  Help understanding error messages in Python.

Example of prompt and interaction:

Part 3: Reflection on Usefulness

The use of generative AI is currently almost mandatory to maximize the quality of a work like this. On my own and without help, I would make certain mistakes when writing in a language that I am proficient in, but far from having a native level.

Code debugging becomes much faster. In my case, I have quite a bit of programming experience, and I usually already know where most of the bugs are going, but in some cases it has been helpful to understand why they could happen.

To understand the theoretical concepts required for the development of the thesis, it worked well to use AI as a teacher to whom I could ask questions. Not simply asking it to explain something, but maintaining a dialogue in which I challenge its answers, explain what I am understanding and what I am not, justify my point of view, etc. This dialogue usually converges to a deeper understanding of the topics than I can reach using only conventional methods.

The downside is that everything it says must be validated, especially if any concept is going to be included in the report. But this also ends up helping my learning.

References

[1] PyTorch, "torch.nn.relu — pytorch 2.7 documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html.

[2] R. Anderson, J. Huchette, W. Ma, C. Tjandraatmadja, and J. P. Vielma, "Strong mixed-integer programming formulations for trained neural networks," *Mathematical Programming*, vol. 183, no. 1, pp. 3–39, 2020.

[3] PyTorch, "torch.optim.adam — pytorch 2.7 documentation," https://pytorch.org/docs/stable/generated/torch.optim.Adam.html.

[4] D. A. Nix and A. S. Weigend, "Estimating the mean and variance of the target probability distribution," in *Proceedings of 1994 IEEE international conference on neural networks (ICNN'94)*, vol. 1. IEEE, 1994, pp. 55–60.

[5] PyTorch, "torch.nn.gaussiannllloss — pytorch 2.7 documentation," https://pytorch.org/docs/stable/generated/torch.nn.GaussianNLLLoss.html.

[6] A. Alcántara and C. Ruiz, "A neural network-based distributional constraint learning methodology for mixed-integer stochastic optimization," *Expert Systems with Applications*, 2023. [En liña]. Dispoñible: https://www.sciencedirect.com/science/article/pii/S0957417423013970

[7] J. R. Birge and F. Louveaux, *Introduction to Stochastic Programming*. Springer, 2011.

[8] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski, *Robust Optimization*. Princeton University Press, 2009.

[9] A. O. Fajemisin, D. Maragno, and D. den Hertog, "Optimization with constraint learning: A framework and survey," *arXiv preprint arXiv:2110.02121*, 2021. [En liña]. Dispoñible: https://www.sciencedirect.com/science/article/pii/S0377221723003405

[10] D. Maragno, H. Wiberg, D. Bertsimas, Birbil, D. den Hertog, and A. Fajemisin, "Mixed-integer optimization with constraint learning," *arXiv preprint arXiv:2111.04469*, 2021. [En liña]. Dispoñible: https://doi.org/10.1287/opre.2021.0707

[11] D. Maragno, H. Wiberg, D. Bertsimas, S. I. Birbil, D. den Hertog, and A. Fajemisin, "Opticl: Mixed-integer optimization with constraint learning," *Optimization Online*, 2023. [En liña]. Dispoñible: https://arxiv.org/abs/2111.04469

[12] A. J. Conejo, E. Castillo, R. Mínguez, and R. García-Bertrand, *Decomposition Techniques in Mathematical Programming*, ser. Springer Optimization and Its Applications. Berlin, Heidelberg: Springer, 2006. [En liña]. Dispoñible: https://link.springer.com/book/10.1007/3-540-27686-6

[13] A. Alcántara, "Distcl: Distributional constraint learning," https://github.com/antonioalcantaramata/DistCL, 2022.